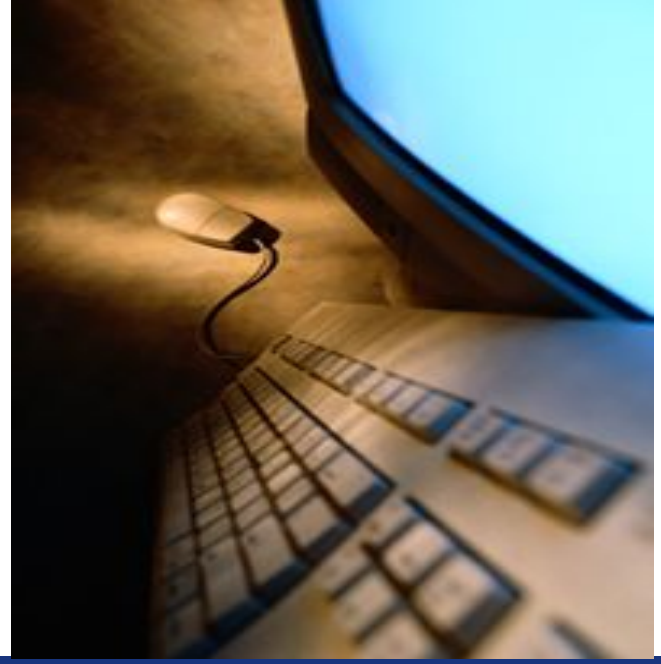
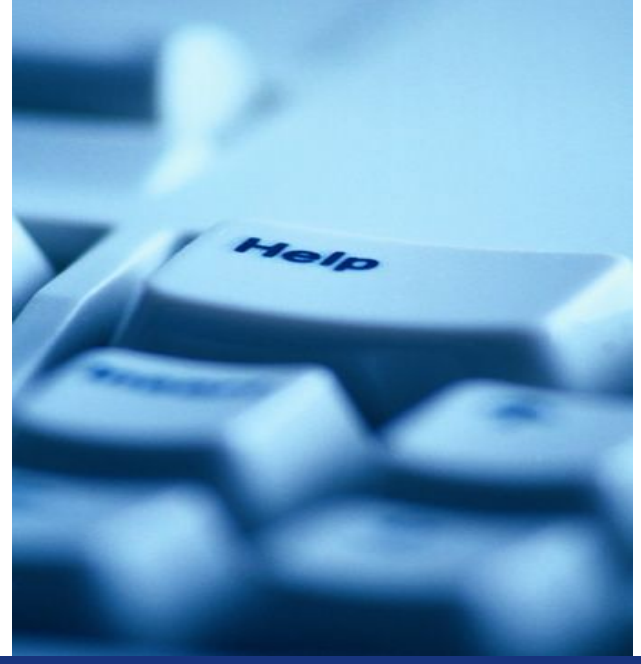
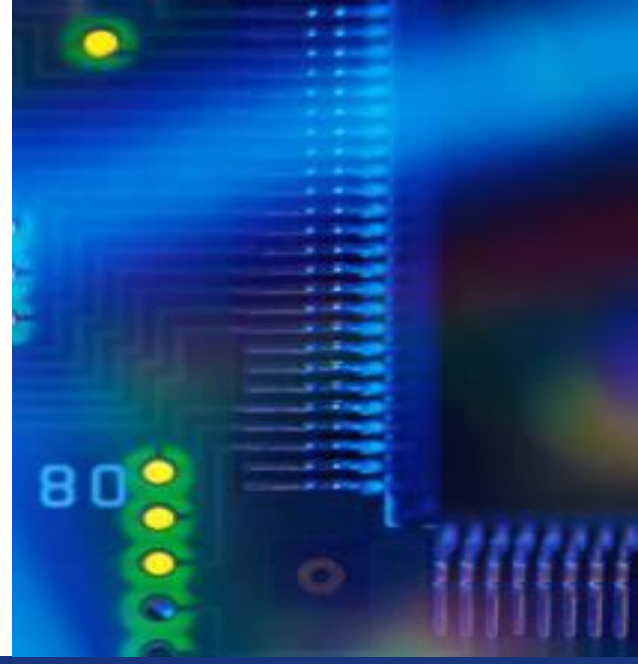
UNIVERSITAS
BUDI LUHUR







FAKULTAS TEKNOLOGI INFORMASI

Pemrograman Java Web dan Enterprise

[PG191/ 3 SKS]

Semester Ganjil 2026/2027

Dosen : Riskiana Wulan S.Kom, M.Kom







Pertemuan 4

Spring Framework: IoC & Dependency Injection






Learning Objectives & Prerequisite

- **Setelah mengikuti perkuliahan ini, mahasiswa mampu:**
  - Memahami permasalahan instantiation object secara manual (new) dan manfaat pengelolaan object terpusat melalui Spring Container.
  - Memahami dan menjelaskan konsep Inversion of Control (IoC) dan Dependency Injection (DI).
  - Membuat dan mengkonfigurasi project Spring Boot menggunakan Spring Initializr (start.spring.io).
  - Menerapkan Spring stereotype annotation (@Component, @Service, @Repository) serta melakukan Constructor Injection untuk mengisolasi dependency antar component.

- **Prerequisite:**
  - Mahasiswa telah menyelesaikan materi Week 3: memahami arsitektur Client-Server, lifecycle HTTP Request-Response, dan pembuatan Servlet dasar berbasis Java Maven.






Big Picture

- **Dari Servlet Manual ke Spring Container**
  - Pada Minggu 3, kita berhasil membuat aplikasi web menggunakan Servlet dan Embedded Tomcat.
  - Namun ketika aplikasi berkembang dengan puluhan class (Controller, Service, Repository, Security, Logger), membuat dan mengaitkan object secara manual menggunakan keyword new akan menimbulkan keterikatan antar-class yang sangat ketat (tight coupling).
  - Spring bertindak sebagai `Container` yang secara otomatis membuat, mengelola, dan menyambungkan object-object di dalam aplikasi melalui konsep `Inversion of Control (IoC)` dan `Dependency Injection (DI)`.

https://spring.io/



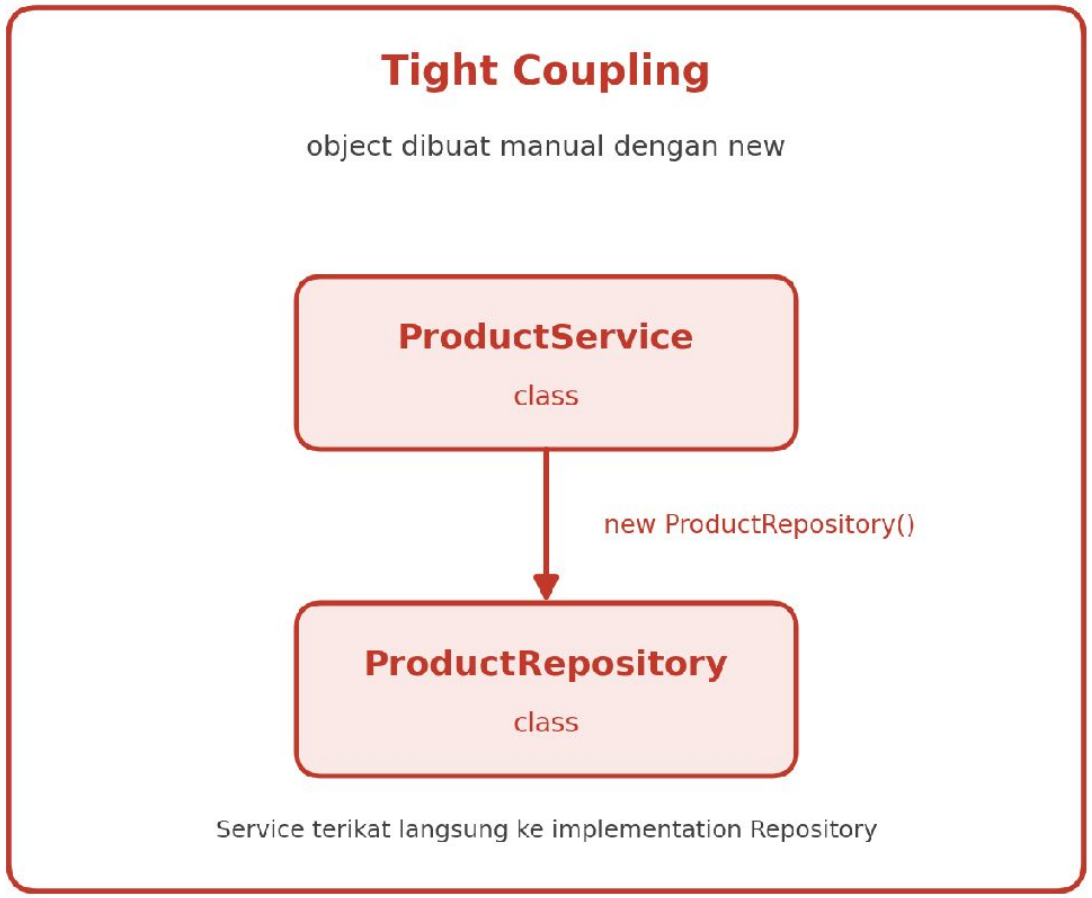


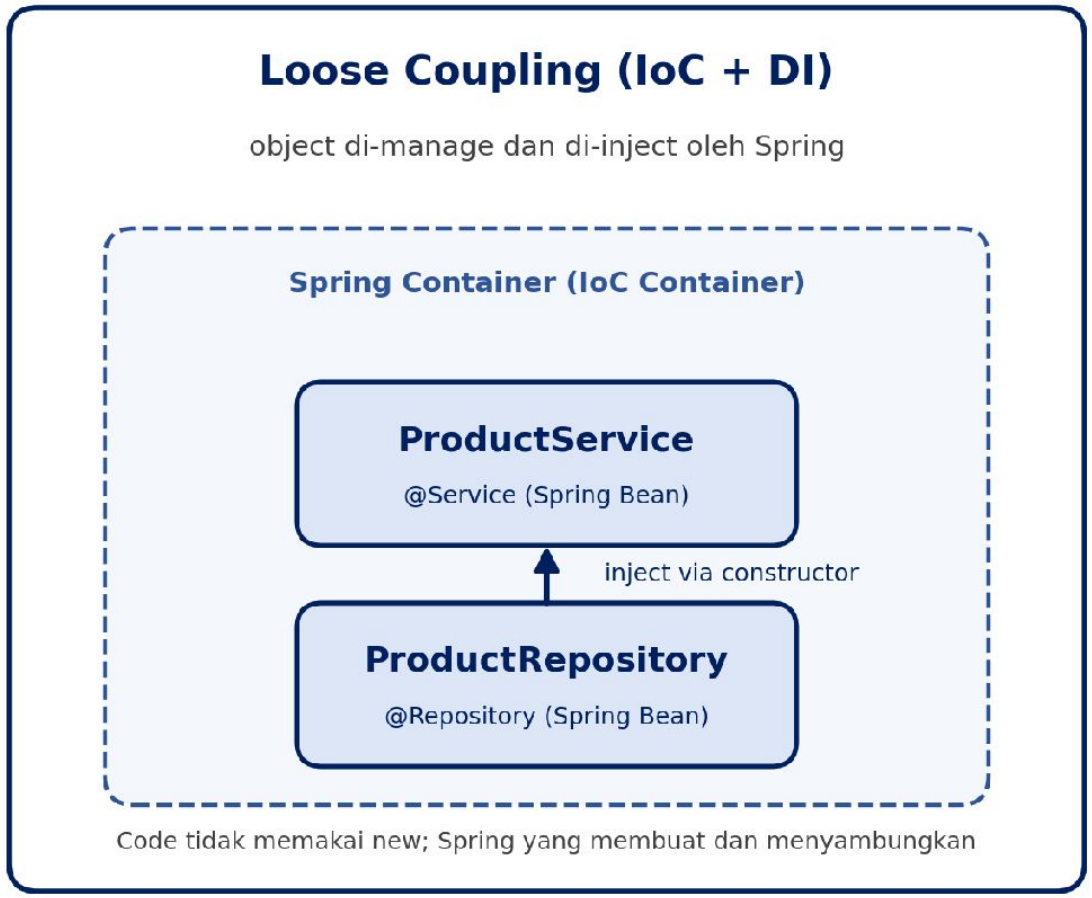

Tight Coupling vs Loose Coupling

- **Instansiasi manual menggunakan new menyebabkan tight coupling antar class.**
  - Jika class ProductRepository memerlukan parameter constructor baru, class ProductService juga harus diubah secara manual.
  - Spring menyelesaikan ini melalui Spring Container (IoC Container) — object dikelola terpusat sebagai Spring Beans.

```
// Contoh Tight Coupling
public class ProductService {
    private ProductRepository
        productRepository =
            new ProductRepository();
}
```

Diagram: Tight vs Loose Coupling









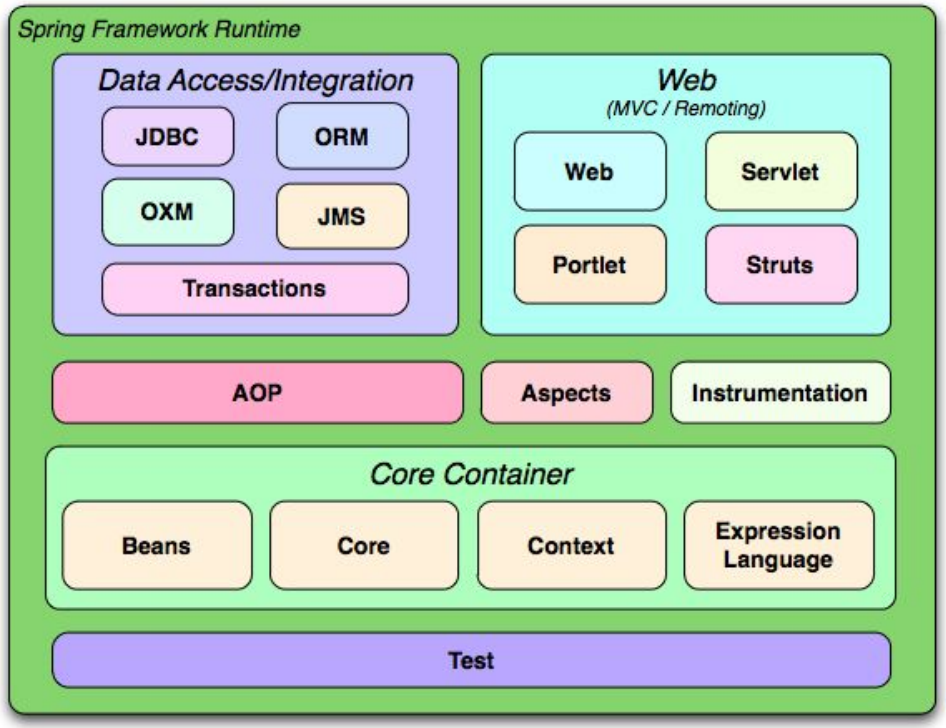

IoC & Dependency Injection (DI)

- **Inversion of Control (IoC): kontrol atas pembuatan, configuration, dan manajemen lifecycle object diserahkan dari code aplikasi ke Spring Container.**
- **Dependency Injection (DI): teknik penerapan IoC di mana Spring Container otomatis meng-inject object dependent yang dibutuhkan suatu class.**
- **Tiga cara melakukan Dependency Injection:**

1. Constructor Injection (rekomendasi utama & standar industri): dependency di-inject lewat constructor. Aman (immutable) dan mempermudah unit testing.
2. Setter Injection: dependency di-inject melalui method setter.
3. Field Injection (annotation @Autowired langsung pada variable): praktis namun kurang disukai untuk clean code enterprise karena menyulitkan testing tanpa Spring Context.

Spring Framework

Spring Framework : Kerangka kerja (framework) berbasis IoC Container yang menyediakan infrastruktur dasar pengembangan aplikasi Java enterprise (terdiri dari modul Core, Data Access/JPA, Web MVC, dan AOP)



- **Core Container (Pondasi Utama)**:
  - Menjalankan **IoC Container / ApplicationContext** untuk mengelola Spring Beans dan menyambungkan dependencies (DI).
- **Data Access / Integration Layer**:
  - Menyediakan abstraksi ORM/JPA (Hibernate) dan manajemen transaksi data (@Transactional).
- **Web Layer (Spring Web MVC)**:
  - Menangani pemetaan HTTP Request, REST Controller (@RestController), dan respon JSON.
- **AOP & Security Layer**:
  - Menangani fungsi lintas-layer (cross-cutting concerns) seperti logging, enkripsi, dan otorisasi.






Spring Boot

Spring Boot merupakan framework yang dibangun di atas Spring Framework untuk menyederhanakan proses pengembangan aplikasi melalui penyediaan fitur **auto-configuration**, **embedded server**, serta fasilitas yang siap untuk lingkungan produksi (**production-ready**).

Arsitektur Spring Boot dirancang secara khusus untuk meminimalkan penulisan kode berulang (boilerplate code), sehingga memfasilitasi pembangunan aplikasi Java yang mandiri (standalone), terukur (scalable), dan mudah dipelihara (maintainable).

Analogi Singkat:

Spring Framework adalah mesin mobil yang sangat bertenaga dan fleksibel.

Spring Boot adalah mobil utuh siap pakai yang sudah terpasang mesin tersebut lengkap dengan starter otomatis.






Komponen Utama Arsitektur Spring boot

Spring Framework

Merupakan fondasi utama (backbone) dari Spring Boot yang menyediakan fasilitas Dependency Injection (DI), Aspect-Oriented Programming (AOP), serta manajemen transaksi (transaction management).

Embedded Server

Aplikasi Spring Boot secara bawaan berjalan di atas web server terintegrasi (seperti Apache Tomcat, Jetty, atau Undertow), sehingga menjadikan bersifat self-contained dan mempermudah proses deployment.

Auto-Configuration

Fitur yang secara otomatis mengkonfigurasi komponen-komponen Spring berdasarkan dependensi library yang terdeteksi di dalam file konfigurasi proyek (pom.xml).

Spring Boot Starter Dependencies

Paket dependensi yang telah terkonfigurasi secara terstandar (pre-configured starter packages) untuk menyederhanakan manajemen library, seperti spring-boot-starter-web, spring-boot-starter-data-jpa, dan sejenisnya.

Spring Boot Actuator

Komponen yang menyediakan fungsi monitoring, evaluasi metrik performance, serta pemeriksaan status (health check) aplikasi pada lingkungan production.






Spring Boot dan Spring Initializr

- **Spring Framework versi klasik memerlukan konfigurasi XML yang rumit. Spring Boot menyediakan Auto-Configuration dan Starter Dependencies.**
- **Project Spring Boot dibuat melalui Spring Initializr (start.spring.io):**
  - Project: Maven
  - Language: Java
  - Spring Boot Version: 4.1.1 (LTS)
  - Packaging: Jar
  - Java Version: 21
  - Dependencies: Spring Web (REST API, Embedded Tomcat, Spring MVC)






Struktur Project Spring Boot

```
product-catalog/
├── pom.xml
├── mvnw / mvnw.cmd / .mvn/
└── src/
    ├── main/
    │   ├── java/com/budiluhur/catalog/
    │   │   └── Application.java
    │   └── resources/
    │       ├── application.properties
    │       ├── static/
    │       └── templates/
    └── test/java/com/budiluhur/catalog/
        └── ApplicationTests.java
```

Keterangan:

- **pom.xml**: dependency dan plugin Maven, termasuk parent spring-boot-starter-parent.
- **mvnw dan .mvn/**: Maven Wrapper, supaya project bisa di-build tanpa instal Maven.
- **Application.java**: main class dengan @SpringBootApplication, harus di root package.
- **application.properties**: adalah file konfigurasi utama di Spring Boot yang digunakan untuk mengatur berbagai pengaturan aplikasi secara eksternal tanpa harus mengubah kode Java
- **static/ dan templates/**: untuk file HTML/CSS/JS dan view template.
- **src/test/**: tempat unit test, memakai spring-boot-starter-test.






Application.properties

Contoh isi file :

```
# Nama aplikasi
spring.application.name=product-catalog

# Port server (default: 8080)
server.port=8081
```

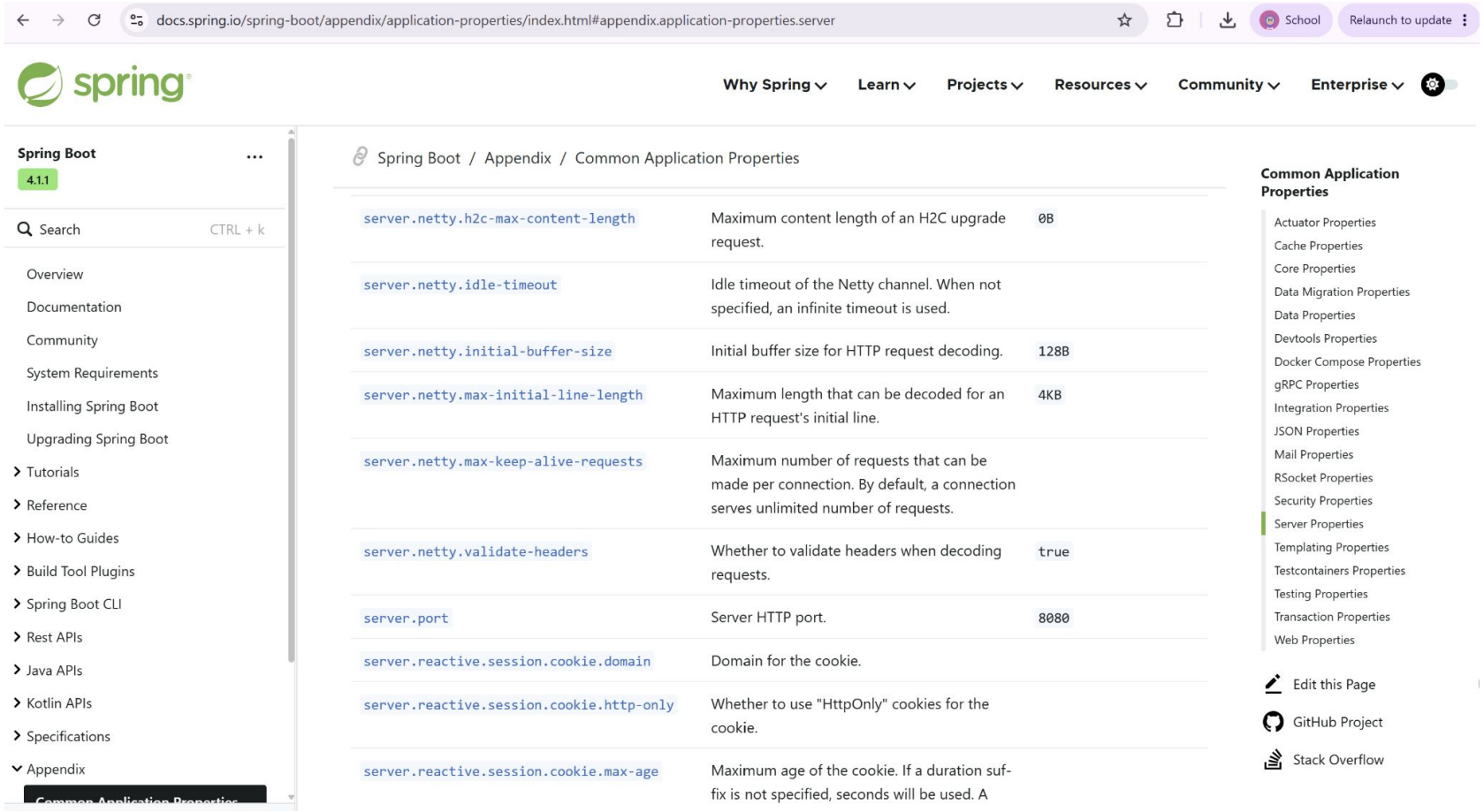
Setelah mengubah file ini, perlu restart aplikasi supaya perubahannya ter-apply.







Application Properties



https://docs.spring.io/spring-boot/appendix/application-properties/index.html#appendix.application-properties.server






Spring Stereotype Annotations

- **Agar Spring Container mengenali kelas Java sebagai Spring Bean, Spring menyediakan anotasi stereotipe:**
  - `@Component`: anotasi umum untuk menandai kelas Java biasa sebagai Spring Bean.
  - `@Repository`: khusus untuk kelas Data Access Object (DAO)/Repositori; menyediakan penanganan exception basis data.
  - `@Service`: khusus untuk kelas yang menyimpan logika bisnis (Business Logic Layer).
  - `@RestController / @Controller`: khusus untuk kelas yang menangani HTTP Request dan merespons data (Web Layer).

Diagram: Spring Layered Architecture









Apa itu Bean?

Bean adalah object yang dibuat dan dikelola oleh Spring Container. Kita tidak membuatnya dengan new. Spring yang membuat object tersebut, menyimpannya, lalu meng-inject-nya ke class lain yang membutuhkan.

Sebuah class menjadi Bean ketika diberi annotation stereotype, misalnya @Component, @Repository, atau @Service. Saat aplikasi start, Spring memindai package dan mendaftarkan semua class yang memiliki annotation tersebut.

Pada project kita:

- ProductRepository (@Repository) menjadi Bean.
- ProductService (@Service) menjadi Bean.
- Spring meng-inject ProductRepository ke ProductService lewat constructor.

Secara default, Spring hanya membuat satu instance untuk setiap Bean (scope singleton), dan instance yang sama dipakai oleh semua class yang membutuhkannya.






Hands-on Lab

- **Tujuan Lab:**

1. Mengubah project product-catalog menjadi aplikasi Spring Boot.
2. Menerapkan Dependency Injection berbasis Constructor Injection.

Step 0 :

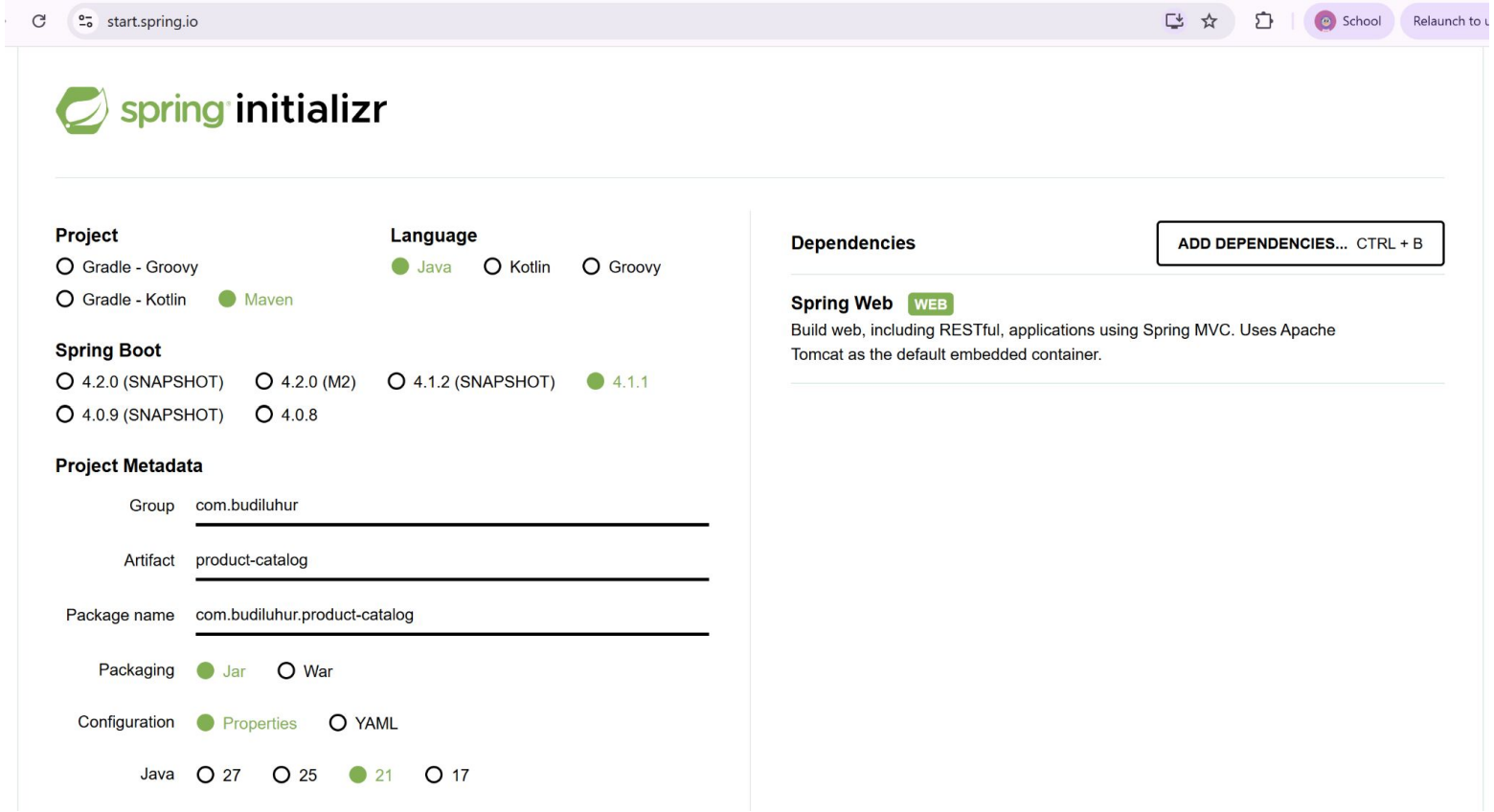
Buat Spring boot project **melalui Spring Initializr (start.spring.io):**

- Project: Maven
- Language: Java
- Spring Boot Version: 3.x.x (LTS)
- Packaging: Jar
- Java Version: 17 / 21
- Dependencies: Spring Web (REST API, Embedded Tomcat, Spring MVC)

Isi Project : Maven
Language : Java

Project Metadata:
Group : com.budiluhur
Artifact : product-catalog
Packaging : Jar
Java : 21

Dependencies:
Spring Web



Lalu Klik Generate dan extract zip
ke lokasi project yang akan digunakan lalu buka menggunakan VSCode






Struktur Project yang akan dibuat

```
src/main/java/com/budiluhur/product-catalog/
├── Application.java
├── exception/
│   └── ProductNotFoundException.java
├── model/
│   └── Product.java
├── repository/
│   └── ProductRepository.java
└── service/
    └── ProductService.java
```

Masing-masing package memiliki fungsi berikut:

- **exception**: menyimpan exception class, misalnya ProductNotFoundException.
- **model**: menyimpan model class yang merepresentasikan data, misalnya Product.
- **repository**: menyimpan repository class yang mengelola akses data, misalnya ProductRepository.
- **service**: menyimpan service class yang berisi business logic, misalnya ProductService.

Step 1: Cek Spring Boot Dependencies (pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>4.1.1</version>
                <relativePath/> <!-- lookup parent from repository -->
        </parent>
        <groupId>com.budiluhur</groupId>
        <artifactId>product-catalog</artifactId>
        <version>0.0.1-SNAPSHOT</version>


  <properties>
                <java.version>21</java.version>
        </properties>
    <dependencies>
                <dependency>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-webmvc</artifactId>
                </dependency>

                <dependency>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-webmvc-test</artifactId
>
                        <scope>test</scope>
                </dependency>
        </dependencies>

    <build>
                <plugins>
                        <plugin>
                                <groupId>org.springframework.boot</groupId>
                                <artifactId>spring-boot-maven-plugin</artifactId
>
                        </plugin>
                </plugins>
        </build>

</project>
```






Step 2: Buat Class ProductRepository

```
package com.budiluhur.product_catalog.repository;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Repository;

import com.budiluhur.product_catalog.exception.ProductNotFoundException;
import com.budiluhur.product_catalog.model.Product;

@Repository
public class ProductRepository {
    private final List<Product> productList = new ArrayList<>();

    public ProductRepository() {
        // Data Awal In-Memory
        productList.add(new Product("PRD-01",
                "Keyboard Mechanical", 450000.0));
        productList.add(new Product("PRD-02",
                "Mouse Wireless Silent", 175000.0));
    }

    public void addProduct(Product product) {
        productList.add(product);
    }

    public List<Product> findAll() {
        return productList;
    }

    public Product findById(String id)
            throws ProductNotFoundException {
        for (Product p : productList) {
            if (p.getId()
                    .equalsIgnoreCase(id)) {
                return p;
            }
        }
        throw new ProductNotFoundException(
                "Produk dengan ID '" + id
                        + "' tidak ditemukan!");
    }

}
```






Step 3: Buat Class ProductService

- **@Service menandai class ini sebagai Spring Bean untuk business logic (Business Logic Layer).**
  - Constructor Injection: Spring otomatis meng-inject ProductRepository saat object ProductService dibuat — tanpa keyword new.
  - Field productRepository dideklarasikan final sehingga bersifat immutable dan lebih aman.

```
package com.budiluhur.product_catalog.service;

import java.util.List;

import org.springframework.stereotype.Service;

import com.budiluhur.product_catalog.exception.ProductNotFoundException;
import com.budiluhur.product_catalog.model.Product;
import com.budiluhur.product_catalog.repository.ProductRepository;

@Service
public class ProductService {
    private final ProductRepository productRepository;

    //constructor injection
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
```






ProductService (Injt.)

```
 public Product getProductById(String id) throws
ProductNotFoundException {
        return productRepository.findById(id);
    }

    public Product createProduct(Product product) {
        productRepository.addProduct(product);
        return product;
    }
}
```






Step 4: Spring Boot Main Class

```
package com.budiluhur.product_catalog;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import com.budiluhur.product_catalog.model.Product;
import com.budiluhur.product_catalog.service.ProductService;

@SpringBootApplication
public class ProductCatalogApplication {

	public static void main(String[] args) {
		SpringApplication.run(ProductCatalogApplication.class, args);
	}

	// Runner sederhana untuk
	// menguji Injection
	@Bean
		public CommandLineRunner run(
				ProductService productService) {
			return args -> {
				System.out.println("=== SPRING BOOT PRODUCT
CATALOG IS RUNNING ===");
				productService.createProduct(
					new Product("PRD-03",
						"Monitor Gaming 24 Inch",
						2100000.0));
				System.out.println("Daftar Produk di Spring
Container:");
				productService.getAllProducts()
					.forEach(System.out::println);
			};
		}
```






Step 5: Running the Application

- **Langkah menjalankan:**

1. run file Application.java dari IDE atau jalankan command berikut pada terminal
   > mvn spring-boot:run
2. Amati konsol terminal — Spring Boot akan menampilkan logo banner Spring, mendeteksi Beans (ProductRepository, ProductService), dan menjalankan CommandLineRunner.

```
[INFO] Attaching agents: []

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

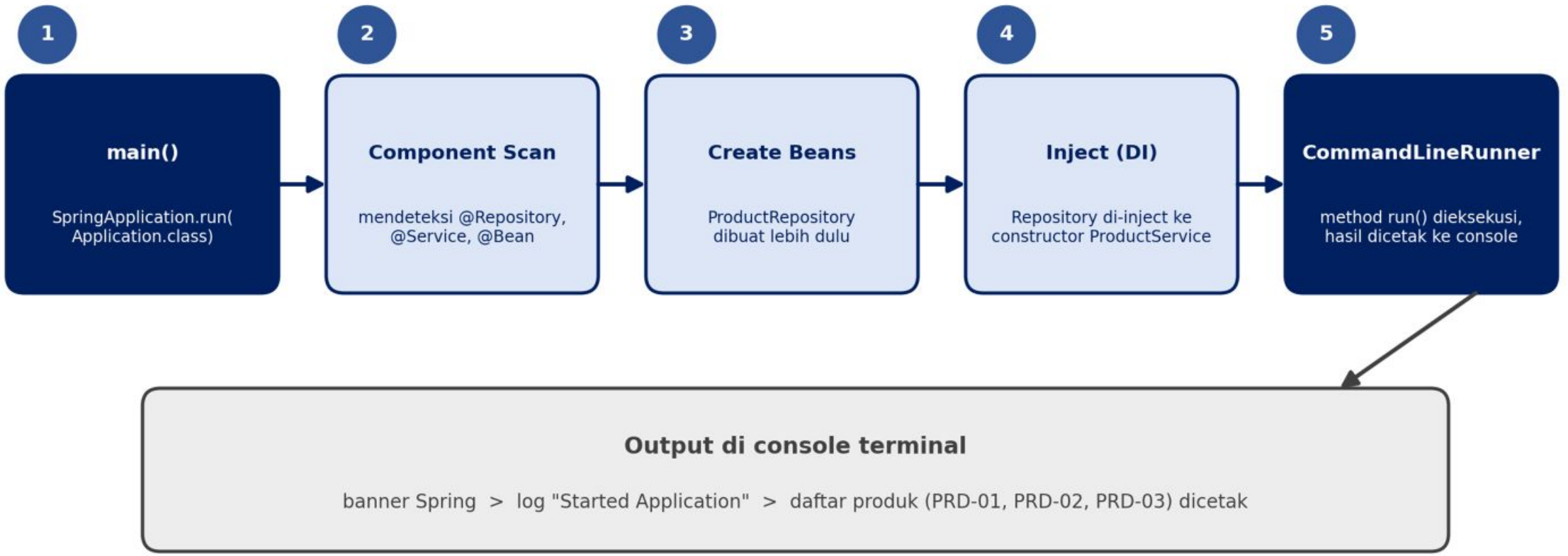
 :: Spring Boot ::                (v4.1.1)

2026-10-11T22:23:23.397+07:00  INFO 2376 --- [product-catalog] [           main] c.b.p.ProductCatalogApplication          : Starting ProductCatalogApplication using Java 21.0.12.1 wit
h PID 2376 (C:\data\github\java-web-enterprise-course\product-catalog\target\classes started by tellr in C:\data\github\java-web-enterprise-course\product-catalog)
2026-10-11T22:23:23.402+07:00  INFO 2376 --- [product-catalog] [           main] c.b.p.ProductCatalogApplication          : No active profile set, falling back to 1 default profile: "
default"
2026-10-11T22:23:24.607+07:00  INFO 2376 --- [product-catalog] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8081 (http)
2026-10-11T22:23:24.628+07:00  INFO 2376 --- [product-catalog] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-10-11T22:23:24.628+07:00  INFO 2376 --- [product-catalog] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-11T22:23:24.722+07:00  INFO 2376 --- [product-catalog] [           main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 125
7 ms
2026-10-11T22:23:25.329+07:00  INFO 2376 --- [product-catalog] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8081 (http) with context path '/'
2026-10-11T22:23:25.331+07:00  INFO 2376 --- [product-catalog] [           main] c.b.p.ProductCatalogApplication          : Started ProductCatalogApplication in 2.613 seconds (process
 running for 3.132)
=== SPRING BOOT PRODUCT CATALOG IS RUNNING ===
Daftar Produk di Spring Container:
[PRD-01] Keyboard Mechanical - Rp450000.0
[PRD-02] Mouse Wireless Silent - Rp175000.0
[PRD-03] Monitor Gaming 24 Inch - Rp2100000.0
```



Diagram: Spring Boot Startup Flow

Alur saat Application.java dijalankan








Mini Challenge

- **Modifikasi project dari Hands-on Lab:**

1. Buat class baru DiscountService dan beri annotation @Service.
2. Tambahkan method public double calculateDiscountedPrice(double originalPrice, double discountPercentage).
3. Inject DiscountService ke dalam ProductService menggunakan Constructor Injection.
4. Buat method baru di ProductService: public double getDiscountedProductPrice(String id, double discount) yang menghitung harga diskon produk berdasarkan ID.






Output

```
=== SPRING BOOT PRODUCT CATALOG IS RUNNING ===

Daftar Produk di Spring Container:
[PRD-01] Keyboard Mechanical - Rp450000.0
[PRD-02] Mouse Wireless Silent - Rp175000.0
[PRD-03] Monitor Gaming 24 Inch - Rp2100000.0

--- INFORMASI HARGA & DISKON ---
[PRD-01] Keyboard Mechanical
- Harga Normal : Rp450.000,00
- Diskon       : 10.0% (Ada Diskon)
- Harga Akhir  : Rp405.000,00

[PRD-02] Mouse Wireless Silent
- Harga Normal : Rp175.000,00
- Diskon       : 0% (Tidak Ada Diskon)
- Harga Akhir  : Rp175.000,00
```






Common Mistakes

- **Problem: Error `NoSuchBeanDefinitionException` saat aplikasi dijalankan — penyebab: lupa menambahkan stereotype annotation (`@Component`/`@Service`/`@Repository`) pada class dependent sehingga Spring Container tidak mendaftarkannya sebagai Spring Bean.**
- **Problem: peringatan IDE "Field injection is not recommended" — penyebab: menuliskan `@Autowired` langsung pada field private. Perbaikan: refactor menjadi Constructor Injection.**
- **Problem: Spring tidak mendeteksi class Bean meski sudah diberi annotation `@Service` — penyebab: class berada di luar package utama `Application.java` (di luar jangkauan `@ComponentScan`). Perbaikan: pastikan seluruh sub-package berada dalam hierarchy yang sama, misalnya `com.budiluhur.catalog.*`.**






Recap & Preparation for Next Week

- **Recap:**
  - **Tight coupling** menyulitkan pemeliharaan dan pengujian aplikasi karena adanya keterikatan instansiasi secara manual (new).
  - **IoC Container** bertugas mengambil alih kendali atas pengelolaan dan penghubungan objek-objek dalam aplikasi Spring.
  - Dependency Injection (DI) menginjeksi kebutuhan objek secara otomatis — *Constructor Injection* merupakan standar terbaik demi keandalan kode.
  - Stereotype annotation utama: `@Repository` (Data Layer), `@Service` (Business Logic), `@RestController` (Web Layer), dan `@Component` (umum).
  - `Spring Boot` mempermudah configuration Spring melalui Auto-Configuration dan Starter Dependencies.
- **Preparation for Next Week:**
  - Minggu 5: Integrasi Konsep Web & IoC/DI untuk Pembuatan REST API Dasar. Praktik menggunakan Spring Boot dan Postman dengan implementasi @RestController, @GetMapping, @PostMapping, @PutMapping, dan @DeleteMapping.



SELESAI